

standard. They naturally assume that whatever the compiler accepts is 'C'. The shock comes later, when they paste their program into an online compiler, an online judge or GCC, and it fails at the very first line: *fatal error: conio.h: No such file or directory.*

The problem is not that a vendor library was used; every platform has its own libraries. The problem is that an entire generation is graduating without ever being told the difference between the C language and one vendor's extensions to it. And there is a further cost: frozen in the DOS era, Turbo C++ cannot interface with any modern third-party library at all, whether for networking, database connectivity or GUI development. That entire world stays invisible to the student.

It teaches facts about computers that are no longer true: Turbo C is a 16-bit compiler, so `sizeof(int)` is 2, and an `int` ranges from -32,768 to 32,767. It exposes the segmented memory model of DOS, with its near, far and huge pointers; no ordinary array or data segment may exceed 64KB, and the whole program is confined within the 640KB conventional memory of DOS, at a time when modern computing routinely handles gigabytes of data. All of this was true once. Today, on every machine our students will use, an `int` is 4 bytes, pointers are 8 bytes, and segmented memory is history. Yet viva questions and textbook margins across the country still confidently state that 'the size of `int` is 2 bytes'. When the lab tool itself is wrong about the machine, students memorise wrong answers and defend them sincerely in examinations and interviews.

Bad habits are never corrected: Modern compilers are strict but helpful teachers. GCC and Clang, invoked with the `-Wall` and `-Wextra` options, warn about uninitialised variables, wrong `printf()` format specifiers, missing return statements and much else. These warnings are free tuition. Turbo C, in contrast, silently accepts `void`

`main()`, tolerates missing prototypes, and knows nothing of the fact that the dangerous `gets()` function was deprecated in C99 and removed from the language altogether in C11. It has no address sanitiser to catch memory errors, and its blue-screen debugger is archaic. A modern IDE lets students set breakpoints, step through code line by line, watch variables change and inspect memory as the program runs, which transforms how a learner understands program execution. For a teacher, this is a classroom aid, not a luxury: place a breakpoint inside a loop, run the program, and the whole class can watch, live, how the loop variable changes on every pass and exactly when a condition fails. Concepts that once needed dry-run tables on the blackboard become directly visible. Students trained on a permissive compiler write careless code and never find out.

The penalty is paid by the students, outside the lab: Every popular coding platform, from CodeChef and HackerRank to LeetCode, compiles submissions with GCC or Clang under current standards; a program carrying `#include <conio.h>` or `void main()` simply will not compile there. Competitive examinations and placement tests assume standard behaviour. Every hour of muscle memory a student builds on Turbo C is an hour of unlearning that the student, and not the institute, will later pay for.

The replacement is free, and it is already the world standard

The replacement does not have to be searched for, evaluated or purchased. It already exists — it is free and open source software (FOSS), and it is the compiler most of the world uses: GCC, the GNU Compiler Collection. A word on the term is in order. FOSS is free of cost, and more importantly, its source code is open for anyone to study, modify and share. It is developed and maintained in public by a worldwide community, which is why GCC has

stayed continuously up to date for nearly four decades while Turbo C froze. It is not freeware of unknown origin, and certainly not a pirated copy; installing it across an entire institution is completely legal and costs nothing. GCC is the default compiler on every Linux distribution and is installed with a single command (`sudo apt install build-essential` on Ubuntu and its derivatives). On Windows, the MinGW-w64 project provides the very same GCC as a native Windows compiler. Clang, from the LLVM project, is the other major FOSS compiler, and is well known for its beginner-friendly error messages. Both track the official standards closely. The essential point for institutes with mixed labs is this: the same compiler, with the same behaviour and the same error messages, is available on Windows and on Linux. A truly common toolchain is possible.

Code::Blocks: A familiar IDE for Windows and Linux

The real attachment of teachers is usually not to Borland's compiler but to the convenience of a single window where one can type a program, press a key and see the output. That convenience is fully preserved. Code::Blocks is a free and open source IDE, released under the GPL, that runs on Windows and Linux (and on macOS as well). It offers a modern editor with syntax colouring and code completion, a Build and Run button, and an integrated graphical debugger, all in one window, in a layout that any Turbo C user will find familiar within an hour.

Setting it up is easier than setting up Turbo C ever was. On Windows, download the Code::Blocks installer that comes bundled with MinGW; this one setup file installs the IDE and GCC together, with nothing further to configure. In practice, this means visiting codeblocks.org, choosing the download where the name contains `mingw-setup`, and clicking through an ordinary Windows installer. The